ISIL

Curso: **Desarrollo de Aplicaciones Empresariales Avanzado**

NRC: **[NRC]**

Proceso de Aprendizaje 2
API de devolución de stock y anulación trazable de pedidos

Docente: **[NOMBRE DEL DOCENTE]**

Estudiante: **Dubois Benites, Jean Franco**

Periodo de estudio: **2026-20**

Repositorio Git: https://github.com/jeanduboiss/pa2-desarrollo-aplicaciones-empresariales

1. Objetivo

Ampliar la integración entre `inventario-service` (Jakarta EE, WildFly, SQL Server) y `sistema-pedidos-springboot` (Spring Boot, H2) con una API REST de devolución de stock, un cliente HTTP desacoplado y capas Controller, Service y Repository administradas por Spring. Se aplican HTTP/JSON y separación de responsabilidades (sesión 4) e IoC/DI con Constructor Injection (sesión 5) en una anulación de pedidos trazable que continúa el caso de PA1. Además se resolvió el reto de compensación.

2. Modificaciones por capa

Proyecto / capa	Cambio
Inventario · dominio	`ProductoInventario.devolverStock(int)`: rechaza cantidad ≤ 0 y suma al stock con `Math.addExact`.
Inventario · servicio	`InventarioService.devolverStockPorCodigo` (`@Stateless`, `REQUIRED`): valida, incrementa y hace `flush()` para traducir un conflicto de `@Version` a HTTP 409.
Inventario · API	`POST /api/inventario/productos/codigo/{codigo}/devoluciones` en `ProductoResource`. DTO `DevolucionStockRequest` (`@NotNull @Positive`) y `DevolucionStockResponse` (`codigo, cantidadDevuelta, stockRestante, version`). Se creó un DTO propio en lugar de reutilizar el de reserva para que ambos contratos evolucionen por separado. 200 / 400 / 404 / 409.
Pedidos · cliente	Interfaz `DevolucionStockClient` e implementación `DevolucionStockRestClient` sobre el `RestClient` existente (bean `inventarioHttpClient`, con timeouts) y la URL `inventario.api.base-url` de `application.properties`. DTO `DevolucionStockRequest/Response`. 4xx/5xx → `InventarioClientException`; conexión o timeout → `InventarioNoDisponibleException`. Sin reintentos del POST.
Pedidos · entidades	`AnulacionPedido` (id, pedidoId, productoCodigo, cantidadDevuelta, fecha, motivo; `UNIQUE(pedido_id)`), enum `PedidoEstado` y `Pedido.estado` (inicial `REGISTRADO`). El pedido nunca se elimina.
Pedidos · repositorio	`AnulacionPedidoRepository extends JpaRepository<AnulacionPedido, Long>` con `findByPedidoId`, `existsByPedidoId` y `findAllByOrderByIdDesc`.
Pedidos · servicio	`AnulacionPedidoServiceImpl` (`@Transactional`): valida pedido y estado → llama al cliente → guarda la anulación → marca `ANULADO`. Constructor con `PedidoRepository`, `AnulacionPedidoRepository` y `DevolucionStockClient`.
Pedidos · controlador	`AnulacionPedidoController`: `POST /api/v1/anulaciones` (201) y `GET /api/v1/anulaciones`; DTO de entrada/salida, valida con `@Valid` y delega en el servicio. `AnulacionPedidoExceptionHandler` devuelve 400 / 404 / 409 / 502 / 503 en JSON.

3. Diagrama actualizado

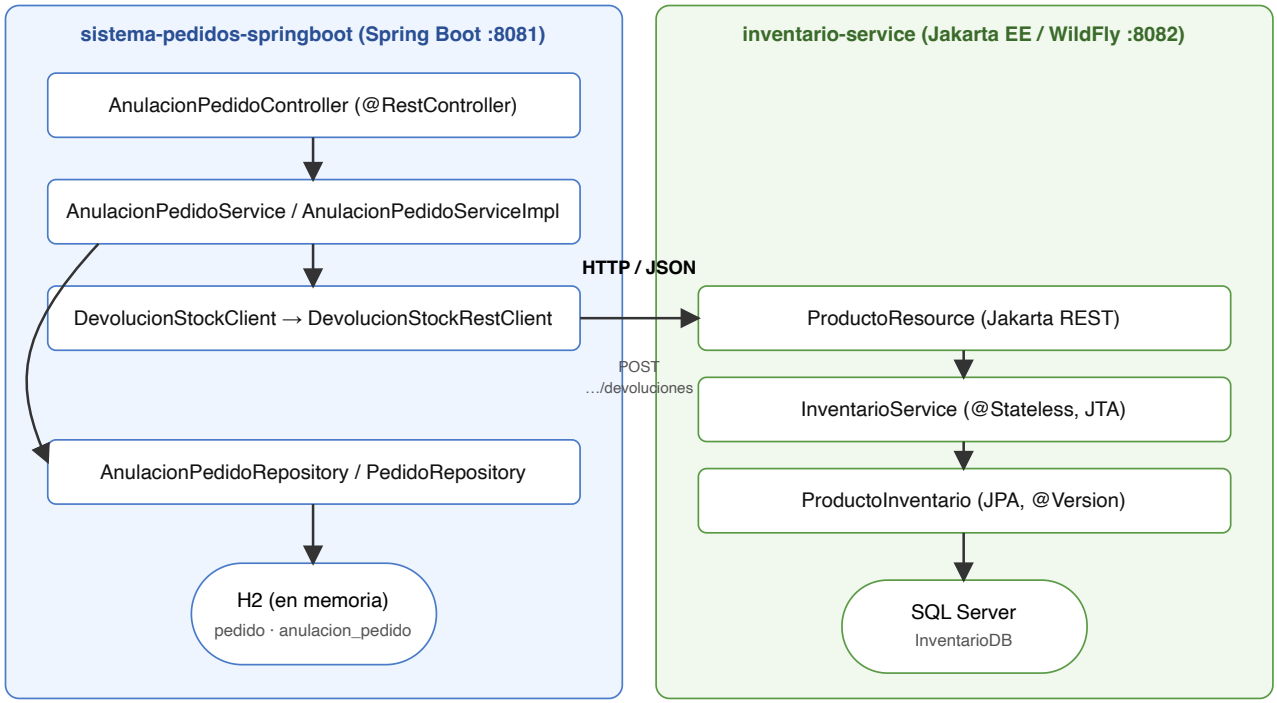


Figura 1. Flujo de anulación: Controller → Service → Client → HTTP/JSON → ProductoResource → InventarioService → SQL Server; Repository → H2.

4. Pruebas unitarias nuevas

Proyecto	Clase de test	Prueba	Resultado
Inventario	ProductoInventarioDevolucionTest	Devolución válida (8 + 2 = 10)	OK
Inventario	ProductoInventarioDevolucionTest	Devolución inválida (0 y –3 rechazadas, stock intacto)	OK
Pedidos	AnulacionPedidoServiceImplTest	Anulación correcta: 1 devolución, estado ANULADO, registro guardado	OK
Pedidos	AnulacionPedidoServiceImplTest	Anulación duplicada: `verify(never())` devolver y guardar	OK
Pedidos	AnulacionPedidoServiceImplTest	Fallo de Inventario: no guarda anulación ni pedido; sigue REGISTRADO	OK

Se añadieron además pruebas extra: una en Inventario (reservar y devolver restaura el stock), pedido inexistente, `CompensacionStockServiceImplTest` (2) y dos clases de integración con H2 real (`AnulacionPedidoIntegrationTest`, 4; `CompensacionReservaIntegrationTest`, 3). `mvn test` en la raíz de cada proyecto: Inventario **5/5** y Pedidos **18/18** en verde (Anexo B). Los tests con mocks validan la lógica del servicio; no demuestran por sí solos un rollback real, por eso se complementan con la integración y las pruebas funcionales.

5. Pruebas funcionales (stack real)

Entorno: SQL Server 2022 (Docker), WildFly 41 con `inventario-service`, Spring Boot en el puerto 8081. Stock inicial de LAP-001: 10. Salidas completas en el Anexo A.

Caso	Solicitud / resultado	Stock y estado
1. Anulación correcta	Pedido de 2 unidades → `POST /api/v1/anulaciones` `{"pedidoId":1,"motivo":"Solicitud del cliente"}` → **201** con `cantidadDevuelta: 2`.	Stock 10 → 8 (reserva) → **10**. Existe `AnulacionPedido`; pedido **ANULADO**.

2. Anulación duplicada	Segundo POST del mismo pedido → **409** "El pedido 1 ya fue anulado."	Stock sigue en 10; 1 solo registro.
3. Pedido inexistente	`pedidoId: 999` → **404**.	Stock sin cambios; sin nueva anulación.
4. Inventario indisponible	WildFly detenido → **503** "…no se encuentra disponible o excedió el tiempo de espera."	Pedido 2 sigue **REGISTRADO**; sin anulación; stock SQL Server 8 (sin cambios).
5. Contrato de la nueva API	`cantidad: 2` → **200** `{"cantidadDevuelta":2,"codigo":"LAP-001","stockRestante":12}`. Cantidad 0, –5 o ausente → **400**. Código inexistente → **404**.	El stock solo cambió con la solicitud válida (10 → 12).

6. Inyección de dependencias: @EJB frente a constructor

En WildFly, `ProductoResource` recibe `InventarioService` con `@EJB` sobre un campo privado: el contenedor Jakarta EE lo inyecta por reflexión después de construir el objeto. La clase no puede crearse con sus dependencias fuera del contenedor, y el campo no puede ser `final`. En Spring, las clases nuevas declaran sus dependencias en el constructor (`AnulacionPedidoServiceImpl` y `AnulacionPedidoController`); el campo es `final`, el objeto nace completo e inmutable y se puede instanciar en una prueba con mocks, sin contenedor. Esto permitió probar el servicio con Mockito sin levantar Spring ni Inventario. Además, el contenedor falla al arrancar si falta una dependencia, en vez de lanzar un `NullPointerException` en ejecución (Broadcom, s. f.; Eclipse Foundation, s. f.-a).

7. Frontera transaccional

La anotación `@Transactional` abre una transacción JDBC sobre H2: solo puede confirmar o revertir los cambios locales (estado del pedido y registro de anulación). La devolución a Inventario es otra transacción, en otra base de datos, dentro de otro proceso; Spring no puede revertirla porque ya se confirmó por HTTP. Por eso la llamada remota se hace **antes** de escribir en H2: si Inventario falla, no queda nada local por deshacer. El riesgo inverso (Inventario confirma y H2 falla) no se elimina; sería necesaria una transacción distribuida o un patrón saga (Garcia-Molina & Salem, 1987; Richardson, 2018). Tampoco se reintenta un POST de devolución: no es idempotente y, si la primera respuesta se perdió, el reintento sumaría el stock dos veces.

8. Reto de compensación (+3)

`PedidoServiceImpl.registrarPedido` reserva stock y luego intenta guardar el pedido. Si algo local falla antes del commit, `CompensacionStockService` llama a la API de devolución y la excepción se relanza para que H2 haga rollback. **Demostración automatizada** (`CompensacionReservaIntegrationTest`, H2 y transacciones reales): el INSERT falla de verdad (cliente de 200 caracteres sobre una columna de 120).

Momento	Inventario (stock)	H2 (pedidos)
Antes	10	0
Tras la reserva confirmada	8	—
Tras el fallo local y la compensación	**10**	**0** (sin pedido confirmado)
Si la compensación también falla	8	0, más 1 `IncidenteConciliacion` pendiente

El incidente se guarda con `REQUIRES_NEW` para que sobreviva al rollback del pedido, y queda marcado para conciliación manual. No se compensa un **timeout** de la reserva: allí el resultado remoto es incierto (Inventario pudo haber descontado o no), y devolver sin saberlo podría dejar el stock inflado.

La compensación solo se ejecuta cuando la reserva está confirmada. No hay endpoints ni controles inseguros: el fallo se provoca únicamente desde la prueba.

9. Conexión con PA1 y limitaciones

PA1, casos 2 y 3. Ajustar la cantidad de un pedido exigiría reservar la diferencia (si sube) o devolverla (si baja); cambiar de producto exigiría devolver el producto anterior y reservar el nuevo. Son dos llamadas remotas coordinadas, con el mismo problema de falta de atomicidad entre sistemas, por lo que requerirían compensaciones o una saga. No se implementaron.

- Devolución remota y escritura local no forman una transacción distribuida: si Inventario confirma y H2 falla al anular, el stock queda devuelto y el pedido en REGISTRADO; reintentar duplicaría la devolución.
- `UNIQUE` y el control de estado cubren el flujo secuencial evaluado; no garantizan idempotencia entre sistemas frente a solicitudes concurrentes.
- La compensación cubre fallos hasta el `flush`; un error exclusivo del commit no se intercepta dentro del método.
- Las pruebas funcionales se ejecutaron con `curl` y consola SQL; sin interfaz JSP nueva (no exigida).

Referencias

Broadcom Inc. (s. f.). *Spring Framework reference documentation*. https://docs.spring.io/spring-framework/reference/

Eclipse Foundation. (s. f.-a). *Jakarta Enterprise Beans*. https://jakarta.ee/specifications/enterprise-beans/

Eclipse Foundation. (s. f.-b). *Jakarta RESTful Web Services*. https://jakarta.ee/specifications/restful-ws/

Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record, 16*(3), 249-259. https://doi.org/10.1145/38713.38742

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Anexo A. Evidencias de los cinco escenarios

Casos 1, 2 y 3

```
=== CASO 1: Anulación correcta ===
  Stock Inventario (SQL Server): 10
> POST /pedidos (registrar pedido, 2 unidades LAP-001)
  HTTP 302 -> http://localhost:8081/sistema-pedidos/pedidos?creado=1
  Stock Inventario (SQL Server): 8
  Pedidos en H2:
   id=1 cliente=Ana LAP-001 x2 estado=REGISTRADO
> POST /api/v1/anulaciones {pedidoId:1, motivo:Solicitud del cliente}
{"id":1,"pedidoId":1,"productoCodigo":"LAP-001","cantidadDevuelta":2,"fecha":"2026-10-
15T15:31:59.772719","motivo":"Solicitud del cliente"}
  HTTP 201
  Stock Inventario (SQL Server): 10
  Pedidos en H2:
   id=1 cliente=Ana LAP-001 x2 estado=ANULADO
> GET /api/v1/anulaciones
[{"id":1,"pedidoId":1,"productoCodigo":"LAP-001","cantidadDevuelta":2,"fecha":"2026-10-
15T15:31:59.772719","motivo":"Solicitud del cliente"}]
  HTTP 200

=== CASO 2: Anulación duplicada ===
> POST /api/v1/anulaciones {pedidoId:1} (segunda vez)
{"status":409,"error":"Conflict","message":"El pedido 1 ya fue anulado.","timestamp":"2026-10-
15T15:31:59.989631-05:00"}
  HTTP 409
  Stock Inventario (SQL Server): 10
> GET /api/v1/anulaciones (sigue 1 registro)
  registros: 1

=== CASO 3: Pedido inexistente ===
> POST /api/v1/anulaciones {pedidoId:999}
{"status":404,"error":"Not Found","message":"No existe un pedido con id 999.","timestamp":"2026-10-
15T15:32:00.197505-05:00"}
  HTTP 404
  Stock Inventario (SQL Server): 10
  registros de anulación: 1
> Validaciones extra: sin pedidoId / JSON inválido
{"status":400,"error":"Bad Request","message":"El pedidoId es obligatorio.","timestamp":"2026-10-
15T15:32:00.415986-05:00"}
  HTTP 400
{"status":400,"error":"Bad Request","message":"El cuerpo JSON de la solicitud es
inválido.","timestamp":"2026-10-15T15:32:00.430406-05:00"}
  HTTP 400
```

Caso 4

```
=== CASO 4: Inventario indisponible ===
> Registrar pedido nuevo (HTTP 302)
Stock en SQL Server (consola SQL):
LAP-001 8 4

(1 rows affected)
Pedidos en H2:
   id=2 LAP-001 x2 estado=REGISTRADO
   id=1 LAP-001 x2 estado=ANULADO
> Deteniendo inventario-service (WildFly)...
  GET inventario -> HTTP 000 (000 = sin conexión)
> POST /api/v1/anulaciones {pedidoId:2, motivo:Prueba sin inventario}
{"status":503,"error":"Service Unavailable","message":"El servicio de inventario no se encuentra
disponible o excedió el tiempo de espera.","timestamp":"2026-10-15T15:32:13.887691-05:00"}
  HTTP 503
Pedidos en H2 (debe seguir REGISTRADO):
   id=2 LAP-001 x2 estado=REGISTRADO
   id=1 LAP-001 x2 estado=ANULADO
> GET /api/v1/anulaciones (solo la del caso 1)
  registros: 1
Stock en SQL Server (sin cambios, 8):
LAP-001 8 4

(1 rows affected)
```

Caso 5

```
### Stock inicial LAP-001
HTTP/1.1 200 OK
{"codigo":"LAP-001","disponible":true,"productoId":1,"stock":10}
### POST devoluciones {cantidad:2} (válida)
HTTP/1.1 200 OK
{"cantidadDevuelta":2,"codigo":"LAP-001","stockRestante":12,"version":1}
### Stock tras devolución
{"codigo":"LAP-001","disponible":true,"productoId":1,"stock":12}
### cantidad 0
HTTP/1.1 400 Bad Request
{"error":"Bad Request","message":"La cantidad debe ser mayor que
cero.","path":"/inventario/productos/codigo/LAP-001/devoluciones","status":400,"timestamp":"2026-10-
15T15:30:49.782198-05:00"}
### cantidad -5
HTTP/1.1 400 Bad Request
{"error":"Bad Request","message":"La cantidad debe ser mayor que
cero.","path":"/inventario/productos/codigo/LAP-001/devoluciones","status":400,"timestamp":"2026-10-
15T15:30:49.818939-05:00"}
### cuerpo sin cantidad
HTTP/1.1 400 Bad Request
{"error":"Bad Request","message":"La cantidad es obligatoria.","path":"/inventario/productos/codigo/LAP-
001/devoluciones","status":400,"timestamp":"2026-10-15T15:30:49.847303-05:00"}
### código inexistente
HTTP/1.1 404 Not Found
{"error":"Not Found","message":"No existe un producto de inventario con código NO-
EXISTE.","path":"/inventario/productos/codigo/NO-EXISTE/devoluciones","status":404,"timestamp":"2026-10-
15T15:30:49.909256-05:00"}
### Stock final (debe ser 12: 10 + 2 válida)
{"codigo":"LAP-001","disponible":true,"productoId":1,"stock":12}
```

Anexo B. Resultados de `mvn test`

inventario-service

```
[INFO] Building inventario-service 1.0.0-SNAPSHOT
[INFO] Running pe.edu.isil.inventario.domain.ProductoInventarioTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.070 s -- in
pe.edu.isil.inventario.domain.ProductoInventarioTest
[INFO] Running pe.edu.isil.inventario.domain.ProductoInventarioDevolucionTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in
pe.edu.isil.inventario.domain.ProductoInventarioDevolucionTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

sistema-pedidos-springboot

```
[INFO] Building sistema-pedidos-springboot 2.0.0-SNAPSHOT
[INFO] Running com.isil.edu.pe.spedidos.integration.AnulacionPedidoIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.192 s -- in
com.isil.edu.pe.spedidos.integration.AnulacionPedidoIntegrationTest
[INFO] Running com.isil.edu.pe.spedidos.integration.CompensacionReservaIntegrationTest
ANTES    -> Inventario stock=10 | H2 pedidos=0
DESPUÉS  -> Inventario stock=10 | H2 pedidos=0 | llamadas=[RESERVAR LAP-001 2, DEVOLVER LAP-001 2]
COMPENSACIÓN FALLIDA -> Inventario stock=8 | H2 pedidos=0 | incidentes=1
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.105 s -- in
com.isil.edu.pe.spedidos.integration.CompensacionReservaIntegrationTest
[INFO] Running com.isil.edu.pe.spedidos.SistemaPedidosSpringbootApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.565 s -- in
com.isil.edu.pe.spedidos.SistemaPedidosSpringbootApplicationTests
[INFO] Running com.isil.edu.pe.spedidos.service.impl.AnulacionPedidoServiceImplTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.427 s -- in
com.isil.edu.pe.spedidos.service.impl.AnulacionPedidoServiceImplTest
[INFO] Running com.isil.edu.pe.spedidos.service.impl.PedidoServiceImplTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.096 s -- in
com.isil.edu.pe.spedidos.service.impl.PedidoServiceImplTest
[INFO] Running com.isil.edu.pe.spedidos.service.impl.CompensacionStockServiceImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 s -- in
com.isil.edu.pe.spedidos.service.impl.CompensacionStockServiceImplTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```